ChatGPT

Current State (Audit vs Campaign)

The **Audit pipeline** and the **Brullama (campaign) pipeline** are now on separate trajectories. On the right side ("Audit"), we ran a 19-item *prospective-surprise* test suite. Initially the audit jobs omitted an explicit `think:false` flag in their expectation of the model's behavior; once we repaired that (ensuring the audit's request representation matches the runner's semantics), the audit completed the replay contract and then began semantic checks. It is now reporting a **rejection population** of examples that violate some semantic invariants (e.g. flagged failures like `failure_correction_direction_reversed` and `introduced_runtime_scope:live`). These varied rejections hint at multiple distinct error mechanisms. On the left side ("Brullama"), we prepared a 40-item run. The environment setup is underway: a container was checked out, environment variables were fixed (PATH corrected), but the final step – building and launching the `llama-server` runner – is still *in progress*. The left-side steps (importing libraries, resolving executables) are complete, and the runner build is producing log output, but no model inference has yet occurred. Repeated "ready" signals ("it's up") have been logged without any new failure, simply indicating the process is still running. In summary:

- **Audit (19-job sample)** – Replay contract now valid (`think:false` included). Several jobs are **semantically rejected** for diverse reasons. The audit's verdicts are now solely driven by semantic checks against each example.
- **Brullama campaign (40-job batch)** – Environment prep nearly complete. Two early failures (import and path issues) have been fixed. **Runner build is ongoing**; no inference jobs have executed yet.

These two lines should remain independent ("different denominators"): do *not* assume audit fixes automatically apply to the campaign or vice versa. If one population predicts something about the other, **cocoon** that prediction and then test it without merging their state.

Joint Assay of Unresolved Distinctions (Dispatch Invariance)

Despite the left side still building, the *surfacer* currently reports `active_work = 0` and `next_automatic_action = NONE`. Formally, this suggests the system is in quiescence: **no further action is currently justified**. But we must verify this by examining whether any *unobserved distinction* could still change that verdict. In other words, we perform a **joint assay** of the remaining uncertainties.

Think of the system's state as a partial information state. For each unresolved variable or observation (e.g. "Is the runner done?"), we ask: *If we learned its true value, could it change the next_action decision?* If any unknown could flip the decision (from "NONE" to "CONTINUE" or vice versa), the state is not truly quiescent. This analysis is akin to checking **non-interference** or **observation-induced branching**: we partition the possible worlds by their admitted consequences, then see if different partitions lead to different legal transitions. Concretely, we consider the **dispatch gate** for lawful execution: it has four outcomes (ADMIT or one of three WITHHOLD reasons). We compute the *consequence quotient* over the candidate states. If

splitting an unresolved variable yields different gate outcomes, that variable is **relevant**. Otherwise it is truly irrelevant. This is similar in spirit to formal bisimulation reduction: a partition of states defines a quotient, and if the partition respects the transition relation, model-checking results carry over【10†L353-L356】. Here the "quotient" is by **observational equivalence w.r.t. the current consequence (C, B, P)**. Only distinctions that actually change the admissible successor set or dispatch outcome must be kept; others can be collapsed without affecting behavior. We find that, at present, no known missing information (e.g. unspecified flags or pending tasks beyond the runner build) changes the next_action. Every conditional dispatch table we compute for each candidate question yields the same `NONE` outcome. Thus **current quiescence holds**: no unresolved distinction alone demands action.

Denominator-Bound Non-Observation Witnesses

Since we have deemed "do not observe any additional information" (DO_NOT_OBSERVE) at this point, we must produce a **proof-carrying witness** certifying that verdict. We enumerate the exact candidate questions we considered (for example, "Is @filename defined?", "Has the model loaded?", etc.), and for each we list all admissible outcomes and the resulting gate decision. This forms a table (denominator-bound to the evaluated set) of the form:

```
Candidate Question O_i:
   possible outcomes → gate decisions
   e.g. {file=a → ADMIT, file=b → ADMIT}
```

When every outcome leads to the same gate decision (`NONE` in our case), the question is irrelevant. We mark that observation as **NONINTERFERING** and gather these tables. This explicit witness data ensures we don't silently overgeneralize: our `DO_NOT_OBSERVE` claim is scoped to the specific questions tested, not all conceivable observations. (This is analogous to how proof-carrying code attaches concrete conditions【10†L353-L356】: the key point is that any *given* question's potential answers have been checked.) Any question that we have not evaluated remains **UNRESOLVED** and blocks a universal claim of inactivity. In practice, unknown fields (like an undefined `@filename`) yield `UNKNOWN` status and thus prevent a blanket `DO_NOT_OBSERVE`. The surfacer correctly keeps them in the "unresolved" bin, preventing premature quiescence.

Joint Observations and Minimal Sufficient Sets

Next, we explore combinations of observations. It is possible (as in many active-learning or information theory problems) that two questions together resolve the decision even if neither does individually. For instance, in an XOR-like scenario each of two binary flags alone may not affect the outcome, but together they could partition the state space into ADMIT vs WITHHOLD. To test this, we examine pairs (or larger sets) of observations and check if their joint outcomes alter the gate. If we find a set $\{O_1, O_2\}$ such that (O1 alone: non-discriminating, O2 alone: non-discriminating) but (O1,O2 together: discriminating), we have identified synergy. We would then update our partition to refine along that joint information.

Eventually, we aim to compute the **Pareto frontier** of observations: a minimal collection of questions that suffices to determine the lawful dispatch. This mirrors the idea of finding a *minimal sufficient information*

state for a given decision policy【15†L70-L78】. According to recent theory on minimal sufficient transition systems, such minimal representations exist (essentially unique up to equivalence) under broad conditions【15†L70-L78】. In this context, each candidate set $S \subseteq \mathcal{O}$ of observations is evaluated for sufficiency: it is **sufficient** if, upon learning all its admissible outcomes, the legal dispatch is fully determined (no uncertainty remains). We then select minimal elements of the set of all sufficient sets. Note that multiple incomparable minimal sets can exist; we do not assume uniqueness. We also account for cost: cheap observations with negligible effect are preferred, but any high-cost observation that is absolutely necessary (cannot be replaced by a cheaper combination) must be included. This is akin to information-value optimization【23†L125-L133】: we might rank observations by "cost per expected reduction in dispatch uncertainty," balancing computational or resource cost against decision impact. In short, we are building an **information frontier**. Only when we cannot find any cheaper observation or smaller set that preserves the dispatch outcome do we finalize our minimal witness set.

Self-Prompt Lineage and Discoveries

We have been self-prompting ("Brudo" style) to drive this analysis. The following lineage summarizes our key self-queries (P0, P1, …) and their outcomes.

- **P0:** *Distinguish waiting vs working.*
- **State:** Left pipeline is building the environment (llama-server build in progress); right pipeline has finished auditing for this stage. The surfacer says `next_automatic_action = NONE` but we see repeated "up" signals from the runner build.
- **Why interesting:** The system is idle per its own state, but an external process is still running. We must avoid confusing "still in progress" with "error."
- **What it caused:** We refrained from dispatching any "fix" to the runner build. We concluded that repeated "it's up" messages simply mean "still running" (no failure state).
- **Reality taught:** The environment build was genuinely still proceeding. No manual intervention was needed. We marked this context as *waiting*, not *broken*, and thus kept `next_action=NONE`.
- **Why it expired:** Once the runner finishes (or an actual model inference occurs), this waiting condition will resolve; until then no new prompt is triggered.

- **P1:** *What must change before an actual inference can run?*

- **State:** No new state except time has passed while the build continues; still `active_work = 0` and `next_automatic_action = NONE`. The unresolved distinction is: "Will the build finish with a runnable model?"
- **Why interesting:** This frames the quiescence condition: the only barrier to work is the unfinished build. If nothing else can change the legal consequences, we should simply wait.
- **What it caused:** We identified that no other missing piece exists beyond the build itself. Therefore, truly **waiting for the build to complete is the only path forward**.
- **Reality taught:** The future event to observe is the completion of the runner build. Until that happens, there is no alternative action to take.
- **Why it expired:** When the build completes (or fails), the assumption behind this question changes. The surfacer will either discover a new ready state or a failure, which will generate the next prompt.

- **(Future P2)** – After the build completes, the next prompt will be automatically generated by the arrival of a new observation (the model runner ready). The content of P2 will depend on the actual outcome (success or failure) of the inference pipeline. The current chain ends in **quiescence** for now.

Prompts that failed productively: N/A so far – neither P0 nor P1 yielded a need for repair or deep fix; they validated waiting.
Prompts generating most information: P0, P1 both confirmed the system's understanding of the situation (waiting vs action).
New questions discovered: "What must change for inference to proceed?" was not explicit until we analyzed P0's outcome. This became P1.
Generalizations killed: We avoided prematurely generalizing that "nothing is broken" from one observation; we scoped our `DO_NOT_OBSERVE` judgment only to the tested factors.
Generalizations survived: The idea that minimal information suffices (from our quotient view) remains intact, supported by theory【15†L70-L78】.
Cross-domain connections: We recognized parallels to **bisimulation quotienting** (abstracting indistinguishable states【10†L353-L356】) and **value-of-information** tradeoffs (observations worth acquiring)【23†L125-L133】.
Model changes: No underlying model or code was altered by these prompts; only our analysis of its output progressed.
Root causes discovered: The "missing wait" was not a defect but normal asynchronous behavior.
Patent topology: No patent algorithm was invoked yet; thus "patent topology changes" are moot.
What the prompt generator learned: The system should treat an ongoing background process as mere waiting. Safety in do-nothing (fail-closed) was the correct approach.
Current self-prompt: None active. We are in a quiescent state waiting on external progress.

In conclusion, we have verified that **no unresolved distinction currently demands action**: the system truly can refrain from observation (DO_NOT_OBSERVE) until the runner build completes. This validates the decision rule: *"do not acquire any further information unless losing it could change the next lawful consequence."*

The next real milestone will be observing the runner build finish. At that point, a new prompt (P2) will be generated automatically. We do not seek Bruce's permission to stop now – our analysis shows that quiescence is warranted until concrete events occur.

CONNECTED SOURCES: We grounded our approach in formal principles of system abstraction and information sufficiency. In formal verification, forming the **quotient** of a transition system via an equivalence relation yields an abstract system whose states represent equivalence classes, inheriting the behaviors of their member states【10†L353-L356】. Similarly, we partitioned possible worlds by their admitted consequences. The theory of **minimal sufficient information states** guarantees that under broad conditions there is an essentially unique minimal representation of the information needed for a policy【15†L70-L78】. Finally, our choice of observations to acquire (or not) is governed by information-value reasoning: the value of a question is zero if all its answers lead to the same action【23†L125-L133】 (justifying `DO_NOT_OBSERVE` when appropriate). These principles underpin the algorithmic decisions we made in analyzing the current state. Each conclusion above is justified by systematically checking partitions of the candidate space (our "witness tables"), in the spirit of proof-carrying abstention.
